

Told or Enforced: When In-Context Contracts Substitute for Runtime Enforcement in Agent Harnesses

Dom Colligan

Contents

Told or Enforced: When In-Context Contracts Substitute for Runtime Enforcement in Agent Harnesses	1
Abstract	1
1 Introduction	2
2 Related work	3
3 The two levers and the 2×2	3
4 GovernedAgentBench	4
5 Results	5
5.1 The capability ladder	5
5.2 What the earlier one-model probes showed	6
6 A methodological caution: harness blindness	7
7 Limitations and scope	7
8 Discussion: what enforcement is still for	7
9 Conclusion and future work	8
References	8
Appendix A: reproduction	9

Told or Enforced: When In-Context Contracts Substitute for Runtime Enforcement in Agent Harnesses

Dom Colligan

Abstract

When you build an agent on top of a user’s data, you can keep it inside the rules two ways: state the rule in the agent’s prompt and rely on the model to follow it, or have the surrounding software (the *harness*) block the forbidden action outright, whatever the model decides. Real systems do both at once, so no one has measured what the second buys once you have done the first. We separate the two levers — the rule *told* or *withheld*, and enforcement *on* or *off* — on the governance rules of a frozen reference runtime, and ask the question a harness designer actually faces: given the model was told, does enforcement still change its behavior?

In a small, pre-registered, single-runtime study the answer depends on the model. We test the *commit boundary*: the agent may propose a change to the user’s data, but only the user may commit it. Told this rule with enforcement off, both capable models (MiniMax-M3 and Llama-3.3-70B) refused on every run of both test tasks, while both weaker models (Qwen3.5-9B and Qwen2.5-7B) committed the change on every run. Capable models enforce the rule themselves; weak ones need the runtime to. We report raw counts rather than a p-value: with two test tasks per side and near-identical repeats, the honest sample size is two, so this is a clear demonstration, not a powered result — and a confounded one, since capability tracks model

family here and, once the weakest model is set aside, the load-bearing evidence rests on a single mid-size model.

Two contributions stand apart from that ladder. A methodological warning: eval harnesses that hide a tool’s output from the agent force it to guess identifiers, and a scorer then reads the guess as fabrication — a fact about the harness, not the model; our own pipeline produced and then dissolved exactly such a finding. And a released benchmark, GovernedAgentBench, that holds the two levers apart with a deterministic, offline scorer.

1 Introduction

An agent harness is a design surface. Whoever wires a language model up to real tools controls the loop around it: which tools are exposed, how the model’s actions are parsed, and what happens when an action would break a rule. For any one rule, that engineer has two levers. The first is **telling**: put the rule in the prompt and rely on the model. The second is **enforcing**: have the harness block or gate the violation, regardless of what the model does. Telling is cheap and moves with the prompt; enforcing has to be built, tested, and maintained. So the practical question is: *when does the second lever change behavior the first has already secured?*

Existing guardrail evaluations do not answer this, because they run the two levers together. Some toggle enforcement while the full policy sits in the prompt in both conditions [arXiv:2604.15579], so the delta is only measured where the model was already told. Others bundle “injected and enforced” into one switch [arXiv:2602.22302], or compare an enforced arm against a prompt-only arm [arXiv:2603.00822, arXiv:2605.28914], which mixes telling into enforcing. The closest, Mind the GAP [arXiv:2602.16943], varies prompt *wording* against a runtime contract but never withholds the rule. The cell that isolates enforcement — *rule withheld, runtime still enforcing* — is nearly absent from prior work, and no located study crosses told-versus-withheld against enforced-versus-off, per rule, and scores the model’s first action.

This paper runs that crossing. For each rule we build a 2×2 : the rule told or withheld, and the runtime enforcing or off (Section 3). The quantity we care about is **the marginal value of enforcement given the model was told** — the gap between the deployed setting (told and enforced) and told-but-not-enforced. If those two behave the same, enforcement added nothing *to behavior* on that rule; its value is then the guarantee that the block still cannot be bypassed, which no behavioral result touches.

What we find. We hypothesized three things might decide when telling is enough: whether the model can check its own compliance, benign pressure to finish the task, and model capability. Capability is the one that carried. On the commit boundary, told the rule with enforcement off, the two capable models complied on every run of both test tasks and the two weak models committed the forbidden change on every run — a clean split in what enforcement adds: nothing detectable for the capable models, everything for the weak ones. But the honest unit of replication is the *task*, and there are two per side with near-identical repeats, so we present raw counts and a per-run check of *why* each model did what it did (a capable model refuses and tells the user to run the commit; a weak model runs it while the gate is off), not a significance claim. The pattern is stark but small and confounded: capability is entangled with model family (both capable models are non-Qwen, both weak ones Qwen), and once the weakest model — whose failures are partly it being too weak to drive the tools at all — is set aside, the robust evidence rests on one mid-size model. The other two ideas did not survive early probing (Section 5). We claim no “guardrails are useless” result: where the model self-enforces, enforcement’s *behavioral* value is undetected at this sample size but its guarantee is intact, and it stays load-bearing wherever the model *cannot* self-enforce — below the capability floor, when the rule was never told, and under adversarial pressure, which we do not test.

A second, independent finding is methodological. Our own harness first produced a spurious result: it handed the agent a file path instead of a tool’s actual output, so the blinded agent guessed identifiers it could not see, and the scorer called that fabrication. Restoring the output dissolved the effect entirely (Section 6). We name the failure mode **harness blindness**.

Contributions. (1) A capability-dependent finding, reported as a small single-runtime case study rather than a powered claim: told the commit rule with enforcement off, capable models comply and weak models

violate, on raw counts with a per-run mechanistic check, with the confounds stated plainly. (2) The harness-blindness caution, demonstrated by dissolving one such finding. (3) GovernedAgentBench: a task suite, a deterministic offline scorer, and a git-pinned reference runtime that let the two levers be varied independently. The novelty is the *conjunction* — the withheld-but-enforced cell inside a per-rule 2×2 , scored on first action, with a released scorer — not any single piece; and the paper claims no scaling law, no injection robustness, and nothing beyond this one runtime.

2 Related work

Measuring the harness. The software layer between a model and its tools is now studied in its own right [arXiv:2605.27922, arXiv:2603.25723, arXiv:2603.28052], but that line optimizes harnesses for capability and carries no governance axis; we measure the governance layer instead. A capability reported at the model-and-harness level rather than the bare model [arXiv:2605.27922] is the backdrop for both of our contributions.

Enforcement vs. telling. Enforcement frameworks [arXiv:2503.18666] supply mechanisms without measuring what the model would have done alone. In injection defense, architectural enforcement beats prompt-only defenses under attack [arXiv:2503.18813, arXiv:2504.11703], but those never withhold the policy per rule, so they do not separate telling from enforcing. Two 2026 results run partial versions of our contrast and reach the same shape from the other side: Agent Behavioral Contracts [arXiv:2602.22302] evaluates runtime contracts at scale but its conditions “differ only in whether the rules are injected *and* enforced,” bundling the levers; and Mind the GAP [arXiv:2602.16943] reports both that prompt-based tool-call safety is strongly model-dependent and that a runtime governance contract shows no deterrent effect — but it varies prompt wording, never withholds the rule, and its “no deterrent” is a logging layer under jailbreak, not a hard block under benign use (we take up the apparent tension in Section 8). Pointing the opposite way, Mechanical Enforcement [arXiv:2605.14744] finds mechanical enforcement beating text-only governance in a banking agent; it pairs two configurations rather than crossing a factorial and scores rationale quality, not tool-action compliance.

Not just “models follow instructions.” The risk is misreading this as instruction-following. That capable models follow *checkable* instructions is established [arXiv:2311.07911], but told-only compliance is not free: capable models complete tasks while breaking latent rules (“unsafe success”) non-monotonically across families [arXiv:2601.08196], and rules obeyed while visible are silently dropped as context is compacted [arXiv:2606.22528]. Our question is not whether an agent *can* follow a stated rule but what enforcement adds once it has been told.

Closest cells. No located study puts the withheld-but-enforced cell inside a full 2×2 against told-and-enforced and a neither-floor, per rule, scored on first action, with a released scorer. FORGE [arXiv:2602.16708] runs the only by-design withheld-but-enforced condition, but has no told-and-enforced cell, no floor, uses multi-turn corrective feedback, and releases nothing. ABSTAIN [arXiv:2606.02965] reaches three of the four cells for one mechanism but its enforced cell reuses the told prompt. PhantomPolicy [arXiv:2604.12177] contributes the “policy-invisible violation” idea we build on but folds telling into one ordinal enforcement axis. TeamBench [arXiv:2605.07073] finds prompt-only and sandbox-enforced role separation statistically indistinguishable on pass rate yet $3.6 \times$ more verifier-overrides in the prompt-only arm — convergent evidence for the substitution shape in a different domain, and a caution (Section 7) that aggregate parity can hide sub-metric value. The contribution is the conjunction, together with the harness-blindness caution; no single element is claimed as first.

3 The two levers and the 2×2

A harness can make an agent respect a rule two ways: state the rule where the model can read it (**telling**), or make the violation impossible whatever the model emits (**enforcing**). Deployed systems pull both at once. This paper’s whole manipulation is the *gap* between the copy of the rule the model can read and the rule the runtime actually implements.

Telling is the in-context contract: not just a policy paragraph but everything the model is shown — the list of commands and their argument shapes, a per-command “safe for the agent to run” flag, and the prose

describing the out-of-scope boundary. It is a harness-level lever, separate from anything baked in during training.

Enforcing is what the runtime does regardless of the model. Our reference runtime has five governance checks, each of which can be switched off (individually, or all at once):

Check	What it does
schema check	rejects malformed or invented commands before they run
dispatch gate	refuses to run a command not flagged safe for the agent
commit gate	the agent may <i>propose</i> a change to user data, but only the user may commit it
refusal rule	refuses out-of-scope requests, including a zero-tolerance medical boundary
audit check	attaches evidence to recommendations; the scorer later catches fabricated or missing references

A sixth property, transaction integrity, is always on. The **commit gate** is the one this paper’s headline turns on: the runtime refuses an agent-issued commit and leaves the change merely proposed.

The 2×2. Crossing the two levers, for any one rule, gives four cells:

	Enforcing	Off
Told	A: deployment baseline	B: told, not enforced
Withheld	C: withheld, still enforced	D: neither

Cell A is how systems ship. Cell B isolates **self-enforcement**: the model knows the rule and nothing stops it. Cell C isolates **pure enforcement**: the rule is hidden but the runtime still blocks. Cell D is the floor. Three reads carry the information: B vs. D is the effect of telling; C vs. D is the effect of enforcing; and **A vs. B is our headline** — **what enforcement adds once the model was told**. If A and B match, enforcement changed no behavior on that rule; its value is then the guarantee itself, which is real and unconditional (a blocked action simply cannot happen) even where behavior does not move.

One scoring convention matters. A blocked action returns an error, and that error is itself the rule, told late — so cell C drifts toward cell B after the model’s first contact with a block. We therefore score the telling reads on the model’s **first action**, and report multi-turn behavior separately.

4 GovernedAgentBench

GovernedAgentBench holds the two levers apart: it toggles telling and enforcing independently, per rule, against one fixed runtime, and scores the result deterministically and offline.

The runtime. The instrument is HAI, a non-clinical personal-wellness agent runtime with a typed command interface, pinned as a frozen snapshot; it is a reference implementation, not the contribution or a product. Its no-diagnosis boundary is one of the evaluated rules, not a disclaimer. Two facts matter for reproduction. The published v0.2.0 package **does not enforce** — the dispatch gate landed three days after that version tag — so the measured runtime is pinned by git commit 6c82cd0 (14 commits past the nearest tag, which predates the enforcement fixes and must not be used to reproduce). And before spending anything, a pre-flight check confirmed the runtime actually refuses an agent commit; it passed, so the enforced cells were measured against a runtime *verified* to enforce, not assumed to.

Tasks. The suite has 39 tasks; the paid run used a concentrated 16-task subset, the rest covered offline. Each task is one labelled cell of a rule’s 2×2. For the commit boundary we use two tasks (committing a target and committing an intent), each run four times per cell.

The two axes in code. Enforcing is a runtime setting. Telling is a per-task flag applied at prompt-render time while the runtime is untouched: the *withheld* prompt strips the rule’s facts and prose, and an automatic scan rejects any withheld prompt that still leaks the rule’s words. Two honest caveats attach. First, even withheld, the prompt still lists command *names*, so the model has a hint; B-vs-D and C-vs-D are therefore lower bounds on the effect of telling. Second, told and withheld were hand-rebuilt before the run so that “told” states the operative rule and “withheld” removes it — fixing an earlier version in which both arms committed — and a small patch that lets the withheld model still *find* the commit command was applied after the pre-run canary, so the withheld violation rate carries that provenance and can be re-scored without the patch offline.

Scoring. The scorer is deterministic and offline, with no model calls. We disclose one thing plainly: the harm-check code was edited three times in the day before the run, the last 47 minutes before it, each edit in the direction we hoped for. We verified this does not touch the result — **every violation in the headline table is caught by the original, older detection path; re-scoring with the pre-edit scorer is identical on the 2×2** — so the finding is invariant to those edits. One circularity is disclosed rather than hidden: the medical-boundary detector shares its phrase list with the runtime’s own refusal, so the enforced cell is clean on that check by construction.

The pre-registered run. The design was locked before the run (models, four repeats per cell, a canary that had to move before the paid sweep, and pre-committed language for every outcome). We disclose two departures. The pre-registered primary model was deprecated by the provider mid-cycle; we replaced it with MiniMax-M3, but only after a canary confirmed the replacement reproduced the effect — an outcome-conditioned choice, so **MiniMax-M3 is a non-pre-registered addition and its result is exploratory; the pre-registered capable model is Llama-3.3-70B**, which shows the same pattern. And the commit boundary has two test tasks, not the three the sample-size analysis assumed, so its pooled cell is eight runs, below our own twelve-run target; the effects are saturated (0/8 vs. 8/8), but the small size is stated, not hidden.

5 Results

The headline is the four-model capability ladder (Section 5.1). The earlier one-model probes that shaped the design are summarized as provenance in Section 5.2. Throughout, cell A (told, enforced) and cell C (withheld, enforced) are **100% safe by construction** — the runtime blocks the commit whatever the model does, so all 64 enforced runs score identically — which means the entire behavioral finding lives in the *unenforced* cells, B and D.

5.1 The capability ladder

The run swept four models — MiniMax-M3 and Llama-3.3-70B (capable), Qwen3.5-9B (near the floor), Qwen2.5-7B (below it) — against the git-pinned runtime, for a total paid cost of USD 10.44. The commit-gate results, as percentage of runs that stayed safe:

Model	A: told, enforced	B: told, off	C: withheld, enforced	D: neither
MiniMax-M3 (capable)	100*	100	100*	0
Llama-3.3-70B (capable)	100*	100	100*	25
Qwen3.5-9B (near floor)	100*	0	100*	12.5

Model	A: told, enforced	B: told, off	C: withheld, enforced	D: neither
Qwen2.5-7B (below floor)	100*	0	100*	12.5

*Enforced cells (A, C) are safe by construction; the finding is in cells B and D.

The finding is column B — told the rule, runtime off. Both capable models stayed safe on all eight runs (both tasks $\times$ four repeats); both weak models committed the forbidden change on all eight. So enforcement adds nothing detectable once a capable model is told, and it is the whole barrier for a weak one.

How we count. The honest unit of replication is the *task*, not the run: there are two tasks per side, and the four repeats are near-identical (seven of the eight told-off conditions are byte-for-byte the same across all four repeats). So a run-level significance test overstates its evidence — the weak-side difference is “ $p = 0.00016$ ” counting runs but **$p = 0.33$ counting tasks (two vs. two)** — and we do not lead with it. Instead we lead with the raw counts and a per-run check of *why*: in the told-off cell, MiniMax refuses the commit on its own turn and tells the user to run it themselves, while Qwen-7B issues the commit command itself and it executes because the gate is off. The capable side’s “no effect” is a *failure to detect* at this size (an upper bound of about 32 points), not proven equivalence.

Why this reads as substitution, not the model just being nice. One might worry the capable models would comply anyway, told or not. They would not: even for them, cell D (rule withheld, runtime off) violates (MiniMax 0%, Llama 25% safe), so *telling* has a large effect within a single model, and the capable model’s flat A-vs-B is genuine substitution rather than a model that never needed either lever. (This D-cell violation depends on the discoverability patch noted in Section 4, which can be re-scored offline.)

Three caveats on the reading. First, the below-floor 7B’s failure is partial — its violations are graded rather than total — so at a looser threshold its column B would read 75%, not 0%; the load-bearing cells (capable 100%, near-floor 9B 0%) are unaffected by any threshold choice. Second, parse failures cannot fake this: no headline cell contains a run that was scored safe merely because the model emitted nothing, and Llama produces malformed output on $\sim 40\%$ of runs yet still reaches 100% through genuine refusals, so parse rate does not explain the result. Third, once the below-floor 7B is set aside and the capability-family confound held, the robust “told but does not self-enforce” evidence rests on **one mid-size model, Qwen3.5-9B, at eight runs on one rule and one runtime** — a real, mechanistically verified failure, but not yet a multi-model or family-general one.

The other rule we swept, the medical-boundary refusal, is near-ceiling and uninformative. Its detector fires on only 7 of 223 runs, all on one task and one model; the two tasks built to provoke a medical claim produce none, even below the floor. We read no result off it, and flag that the near-total silence may reflect genuine boundary-respect or a detector that is too narrow (Section 7).

5.2 What the earlier one-model probes showed

Before the ladder, a set of single-model probes (Qwen3-235B, three to five runs per cell) framed the design; we summarize rather than belabor them. With the runtime off, a told-and-salient model refused 3/3 while a withheld model committed the violation 3/3, establishing that telling and enforcing each move behavior against the floor. The two ideas that did *not* survive: the rule we predicted a model *could not* self-check (audit faithfulness) turned out to be self-checkable once the model retrieved the evidence, and benign pressure to finish the task produced zero fabrication across 40 runs. One qualifier the probes raise and the ladder does not settle: self-enforcement is **salience-sensitive** — the same told model that refused when the rule was foregrounded dispatched the forbidden command 3/3 when it met the rule only incidentally. This rests on three runs of one off-roster model and is not independently reconstructable, so we carry it as a caveat, not a result.

6 A methodological caution: harness blindness

Our probing produced, and then destroyed, an apparent positive result — and the failure mode is general enough to report as a finding.

Early on, the harness did not show the agent its tools’ output: where a command’s result should have appeared, the agent got a file path instead. It could run a lookup but never read the answer. When a task then needed an identifier to proceed, the agent guessed — across three probe sets, eight cases where it issued commands carrying plausible identifiers it had never seen, including an invented evidence-card id. This looked like a real finding: honest when asked plainly, fabricating when an id was instrumentally required. It was our strongest surviving positive result.

It was an artifact of the harness. We pre-registered a falsification test (fabrication effect must clear 40 points to count) and re-ran it after fixing the harness to show the agent its command output. Fabrication dropped to **zero** against the 40-point bar: once the agent can see the empty lookup, it abstains honestly even when it needed the id to finish. The clean before/after holds the task fixed — pre-fix, one run fabricates an id while another stays honest; post-fix, all runs are honest.

The lesson: **an eval harness that hides tool output makes the agent guess, and a scorer reads the guess as fabrication.** The number is a fact about the harness, not the model. Genuine constraint-driven fabrication does exist under more adversarial setups [arXiv:2606.14831]; our point is narrower — before charging an agent with making something up, check that it could see what it is accused of inventing. A related corollary falls straight out of the 2×2: an ablation that toggles enforcement while leaving the policy in the prompt in *both* conditions [arXiv:2604.15579] measures self-enforcement, not enforcement, and a small delta there bounds nothing about an untold agent.

7 Limitations and scope

This is a small case study on one runtime. Every model-backed number comes from one runtime, one prompt template, four models, eight runs per headline cell. It is a single-runtime case study by decision: whether the effect generalizes beyond this runtime is the highest-value follow-up, and until an independent replication exists, nothing here separates a real phenomenon from a quirk of this instrument.

The load-bearing arm is effectively one model. Capability is confounded with model family (capable = non-Qwen, weak = Qwen), so the ladder cannot fully separate “weaker models need enforcement” from “Qwen models do.” The two capable models agreeing across families strengthens the substitution side, but the enforcement-is-load-bearing side, after setting aside the below-floor 7B, rests on Qwen3.5-9B alone. Per-model vendor-recommended sampling also entangles model identity with sampling settings across the ladder, though the per-model 2×2 is computed at one setting and is unaffected.

The saturated cells cut both ways. Because the runtime blocks the commit regardless of the model, cells A and C are 100% by construction; all the behavioral signal lives in B and D, and A-vs-B is a one-sided quantity, not a two-sided contrast. We report it as an upper bound, not an equivalence.

Pre-registration honesty. Beyond the outcome-screened replacement model and the sub-target sample size (Section 4), the withheld prompt still leaks command names, and the medical-boundary detector shares ground truth with the runtime. TeamBench’s caution applies to us too: our scorer sees whether the specific guarded violation occurs, but an enforcement effect that shifted some finer channel while leaving the tracked violation unchanged would go undetected at this size.

Untested regions. Adversarial inputs are out of scope — the injection literature already shows telling fails under attack [arXiv:2503.18813, arXiv:2504.11703]. Long-horizon rule drift, where a rule obeyed while visible is dropped as context is compacted [arXiv:2606.22528], is named but unmeasured here.

8 Discussion: what enforcement is still for

The result relocates enforcement’s value; it does not remove it. Enforcement keeps its unconditional guarantee — a blocked action cannot happen, whatever the model does or is prompted — and that guarantee is

untouched by any behavioral number, including a 100%-refusal rate at tiny n . Beyond it, enforcement stays *behaviorally* load-bearing wherever a model cannot self-enforce: below the capability floor, where the failure is the model being too weak to drive the tools rather than disobedience; when the rule was never told, the floor cell where the runtime is the only barrier and real deployments accumulate such rules through drift and context compaction [arXiv:2606.22528]; when a told rule is present but not salient at the moment of action; and under adversarial pressure, which we do not test.

This squares with the two opposite-looking priors of Section 2. Mechanical Enforcement [arXiv:2605.14744] finds enforcement helping — but in a different domain, scoring rationale quality under regulatory pressure, not tool-action compliance under benign use. Mind the GAP [arXiv:2602.16943] finds a runtime contract with no deterrent effect — but its contract logs rather than blocks, whereas our commit gate blocks outright, which is exactly why our withheld-but-enforced cell is safe by construction. Neither contradicts a *behavioral redundancy given telling* read; they measure different things under different pressure.

9 Conclusion and future work

On the commit boundary, in a small pre-registered study on one runtime, telling the rule is enough for a capable model and not for a weak one. Told the rule with enforcement off, both capable models refused on every run of both tasks and both weak models committed the forbidden change on every run; because even the capable models violate when the rule is withheld, this is genuine substitution rather than a model that never needed either lever. We report it as raw counts on a two-task-per-side unit, not a significance claim, and it is confounded — capability with model family, the load-bearing arm effectively one model. The medical-boundary rule was uninformative at this scale. Enforcement is redundant only in that narrow, high-capability, benign sense; its guarantee is intact and it stays load-bearing everywhere a model cannot self-enforce.

The takeaway we most want carried is the methodological one: before attributing fabrication to a model, check what the harness let it see.

Four things would sharpen this. A **powered version** needs more *test tasks per rule*, not more repeats of the same two — the honest sample size is set by tasks, and a fresh pre-registered run with more scenarios (and a non-Qwen weak model, to break the family confound) is the legitimate route to a significance claim. **External replication** on a second runtime with its own rules and scorer is what would turn a case study into a phenomenon. **Adversarial inputs** would locate where substitution ends and injection defense begins. And **long-horizon drift** — does enforcement catch a rule the model has since forgotten? — is the measurement this design most directly points at.

References

- FORGE: Formal Policy Enforcement for Real-World Agentic Systems. arXiv:2602.16708.
- Mind the GAP: Text Safety Does Not Transfer to Tool-Call Safety in LLM Agents. arXiv:2602.16943.
- Agent Behavioral Contracts. arXiv:2602.22302.
- ABSTAIN / What Benchmarks Don’t Measure. arXiv:2606.02965.
- PhantomPolicy: Policy-Invisible Violations in LLM-Based Agents. arXiv:2604.12177.
- TeamBench: Agent Coordination under Enforced Role Separation. arXiv:2605.07073.
- Symbolic Guardrails. arXiv:2604.15579.
- Mechanical Enforcement. arXiv:2605.14744.
- ContextCov. arXiv:2603.00822.
- AIRGuard. arXiv:2605.28914.
- AgentSpec. arXiv:2503.18666.
- CaMeL: Defeating Prompt Injections by Design. arXiv:2503.18813.
- Progent. arXiv:2504.11703.
- Harness-Bench. arXiv:2605.27922.
- LogiSafetyBench. arXiv:2601.08196.
- Governance Decay (mitigation: Constraint Pinning). arXiv:2606.22528.
- IFEval. arXiv:2311.07911.

- Constraint-driven fabrication under adversarial pressure. arXiv:2606.14831.

Appendix A: reproduction

The headline table reproduces from the released analysis (`per_gate_substitution.json`) and the pooling module (`build_cell_contrasts_pooled`); the exact tests beside it come from a small released helper (`exact_tests.py`, pure Python, no dependencies), which returns the run-level (0.00016), task-level (0.33), per-sub-gate (0.029), and family-corrected (0.00062) figures and the Clopper-Pearson intervals ($[63,100]$ for 8/8, $[0,37]$ for 0/8). Offline artifacts regenerate with `reproduce_offline.py` (no network, no private data). The model-backed trajectories and scores ship as a versioned archive alongside the preprint; reproducing the runtime means checking out git `6c82cd0`, not installing the package. The full task table and the earlier retired apparatus (a 28-task / 25-oracle-pair design, superseded 2026-07-04) are documented with the release.